Yes. For MAANG+ preparation, the most useful way to understand an LLM is to treat it as **a distributed probabilistic program built around a transformer**, and then study it at three abstraction levels:

1. **DSA:** tensors, matrices, attention, tokenization, sampling, KV cache, search/decoding algorithms.
2. **LLD:** classes/components, interfaces, data flow, model layers, inference engine, tokenizer, sampler, cache, tool/function calling.
3. **HLD:** training infrastructure, model serving, distributed inference, GPU/TPU architecture, data pipelines, model registry, RAG, agents, observability, reliability, and cost.

Below is a systems-engineering view that connects all three.

---

1. What exactly is an LLM?

An **LLM (Large Language Model)** is fundamentally a neural network that learns a probability distribution over token sequences.

Given tokens:

$$
x_1,x_2,\ldots,x_n
$$

the model estimates:

$$
P(x_{n+1}\mid x_1,x_2,\ldots,x_n)
$$

During generation, it repeatedly predicts the next token:

```
"The capital of France is"
            ↓
        P(token)
            ↓
         Paris
            ↓
"The capital of France is Paris"
            ↓
        P(token)
```

```
                ↓
                "."
```

So at the mathematical core:

```
Text
  ↓
Tokenizer
  ↓
Token IDs
  ↓
Embeddings
  ↓
Transformer
  ↓
Logits
  ↓
Probability distribution
  ↓
Sampling / decoding
  ↓
Next token
```

The important realization for system design is:

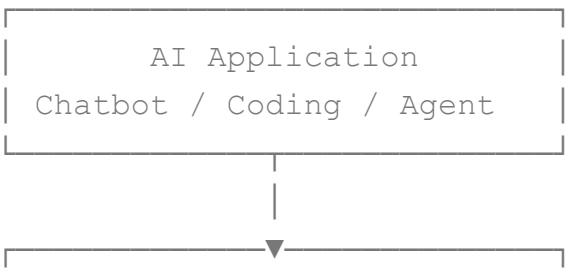
> **An LLM is not fundamentally a database, search engine, knowledge graph, or reasoning engine. It is a parameterized function that maps a sequence of tokens to a probability distribution over possible next tokens.**

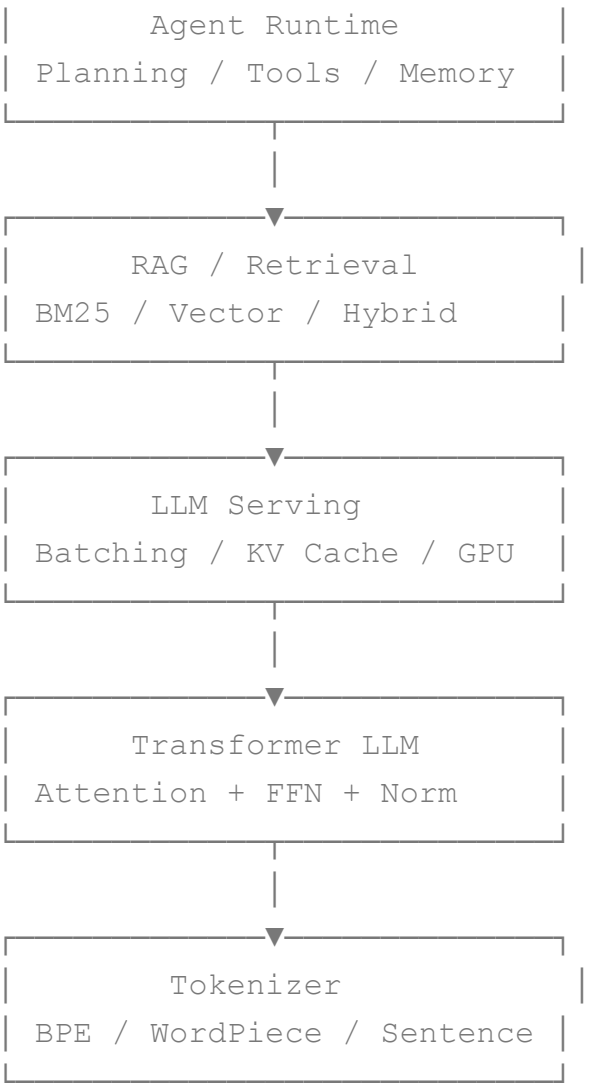
Everything else—RAG, agents, tools, memory, guardrails, orchestration—is built around that core.

---

2. The complete LLM stack

Think of the stack like this:





For interview preparation, you should understand every layer.

---

3. DSA perspective

The DSA perspective answers:

> **What algorithms and data structures make an LLM work?**

The most important ones are:

- arrays/tensors
- matrices
- hash maps
- trees
- graphs
- heaps
- queues
- dynamic programming

- probability distributions
- numerical algorithms
- approximate nearest-neighbor search
- caching
- parallel algorithms

---

4. Tokens and tokenization

Before the model can process text, text is converted into tokens.

For example:

```
"unbelievable"
```

could become:

```
["un", "believ", "able"]
```

and eventually:

```
[421, 8721, 312]
```

The exact tokenization depends on the tokenizer.

Modern LLMs commonly use variants of subword tokenization such as:

- BPE
- WordPiece
- Unigram/SentencePiece-style approaches

---

5. Why tokenization is a DSA problem

A tokenizer performs operations such as:

```
string
 ↓
candidate pieces
```

```
  ↓
merge / segmentation
  ↓
token IDs
```

Efficient implementations rely on data structures such as:

```
HashMap
Trie
Priority Queue
Arrays
```

For example, a vocabulary can conceptually be represented as:

```
root
 ├── a
 │   ├── b
 │   └── c
 ├── b
 │   ├── e
 │   └── i
 └── c
```

A trie provides efficient prefix lookup.

---

6. Embeddings

After tokenization:

```
Token ID
   ↓
Embedding lookup
   ↓
Vector
```

Suppose vocabulary size is:

$$
V=100,000

$$

and model dimension:

$$
d=4096
$$

Then the embedding matrix is:

$$
E \in \mathbb{R}^{100000\times4096}
$$

For token `Paris`:

```
token ID = 18721
```

the model effectively retrieves:

```
E[18721]
```

which is a 4096-dimensional vector.

This is essentially an **array lookup**.

---

7. Tensors

At the DSA level, tensors are the fundamental data structure.

A tensor is a multidimensional array.

For example:

```
Batch × Sequence × Hidden
```

might be:

```
32 × 2048 × 4096
```

Meaning:

```
32 requests
×
2048 tokens
×
4096 dimensions
```

The entire Transformer operates primarily on tensors.

---

8. Matrix multiplication

This is arguably the most important computational operation in an LLM.

Given:

$$
A_{m\times k}
$$

and

$$
B_{k\times n}
$$

we compute:

$$
C=AB
$$

where:

$$
C_{ij}=\sum_{k}A_{ik}B_{kj}
$$

LLMs perform enormous numbers of matrix multiplications.

This is why GPUs are so important.

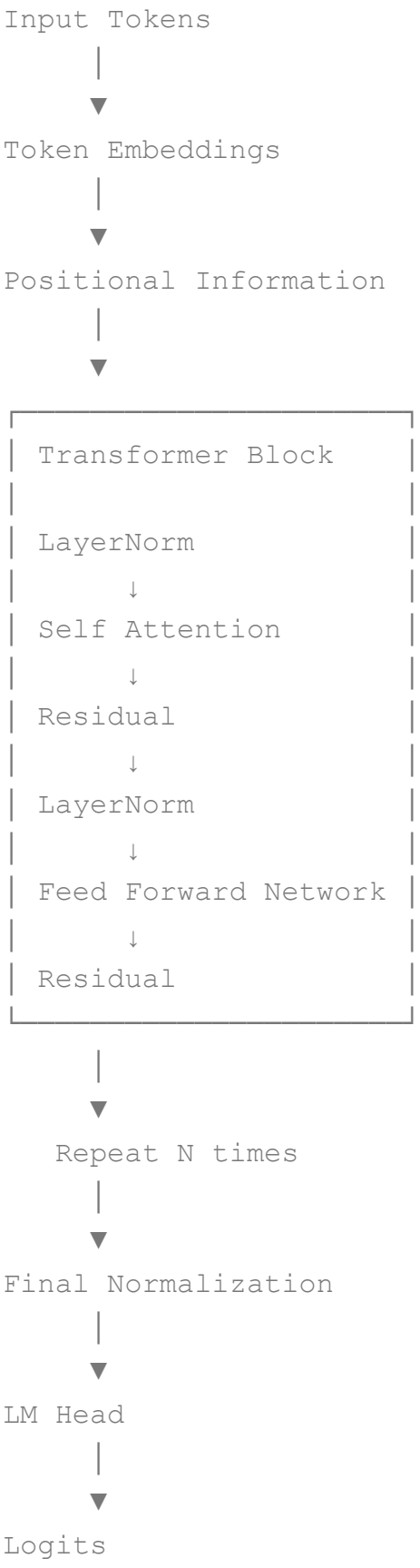
GPUs are highly optimized for parallel matrix operations.

---

9. Transformer architecture

The core architecture is the **Transformer**.

A simplified decoder-only Transformer:



10. Self-attention

This is one of the most important algorithms to understand for interviews.

Given input matrix:

$$
X
$$

we compute:

$$
Q=XW_Q
$$

$$
K=XW_K
$$

$$
V=XW_V
$$

where:

- Q = Query
- K = Key
- V = Value

Then:

$$

Attention(Q,K,V)

softmax
\left(
\frac{QK^T}{\sqrt{d_k}}
\right)V
$$

This single equation is worth knowing extremely well.

---

11. Why Q/K/V?

Consider:

```
"The cat sat on the mat because it was tired."
```

When processing:

```
"it"
```

the model needs to determine which previous words are relevant.

Attention calculates relationships between tokens.

Conceptually:

```
it
 ├── cat         0.52
 ├── sat         0.08
 ├── mat         0.17
 ├── tired       0.18
 └── because     0.05
```

These numbers are attention weights.

The actual model operates on vectors rather than semantic labels, but this illustrates the idea.

---

12. Attention as a matrix operation

Suppose:

```
Sequence length = N
Head dimension = d
```

Then:

```
Q = N × d
K = N × d
```

Therefore:

```
Q × K^T
```

produces:

```
N × N
```

matrix.

This means every token can potentially interact with every other token.

Therefore:

$$
O(N^2)
$$

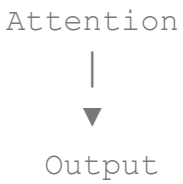
attention complexity with respect to sequence length.

This is one of the most important LLM scaling problems.

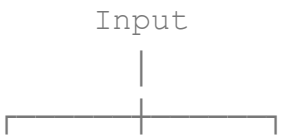
---

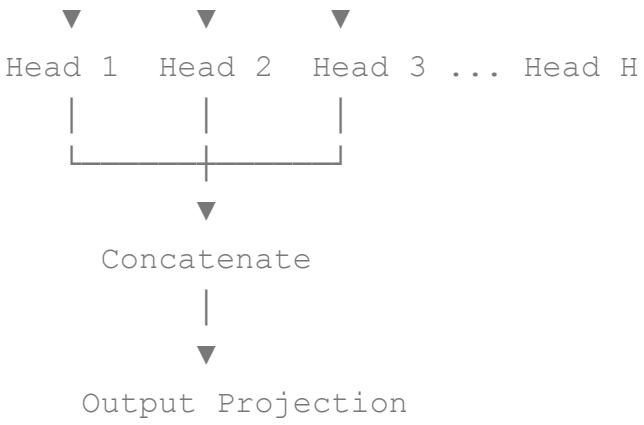
13. Multi-head attention

Instead of having one attention mechanism:



we have multiple heads:





Different heads can learn different relationships.

---

14. Causal attention

For autoregressive language generation, a token cannot look into the future.

Example:

```
I love machine learning
```

When predicting:

```
machine
```

the model can see:

```
I love
```

but not:

```
learning
```

This is implemented using a causal mask:

```
          I love machine learning

I         ✓    ×       ×       ×
love      ✓    ✓       ×       ×
```

```
machine ✓    ✓       ✓       ×
learning✓    ✓       ✓       ✓
```

This is essentially a triangular matrix.

---

15. Feed-forward network

Each Transformer block also contains an FFN.

A simplified version:

$$
FFN(x)=W_2\sigma(W_1x+b_1)+b_2
$$

Modern architectures often use gated activations such as:

```
SwiGLU
```

The FFN typically expands the hidden dimension:

```
4096
  ↓
11008
  ↓
4096
```

This consumes substantial compute and parameters.

---

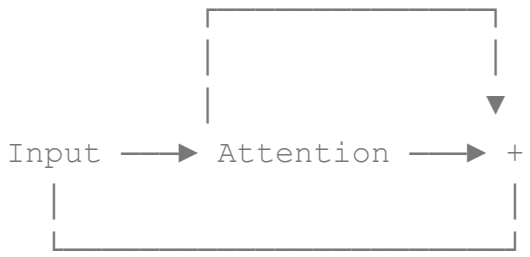
16. Residual connections

A Transformer block doesn't simply replace the input.

It uses:

$$
y=x+f(x)
$$

Conceptually:



This helps optimization and gradient flow.

---

17. Normalization

Transformers use normalization mechanisms such as:

```
LayerNorm
RMSNorm
```

For example, RMSNorm normalizes based on root mean square.

Normalization helps keep activations numerically stable.

---

18. Positional information

Self-attention itself does not inherently understand ordering.

These two sequences:

```
Dog bites man
```

and:

```
Man bites dog
```

contain the same tokens but have different meanings.

Therefore positional information is introduced.

Common approaches include:

- learned positional embeddings
- sinusoidal embeddings
- RoPE — Rotary Position Embedding

RoPE is especially important in modern LLMs.

---

19. Logits

At the end of the Transformer:

```
Hidden vector
      ↓
Linear projection
      ↓
Vocabulary-sized vector
```

Suppose vocabulary size is:

```
100,000
```

The output is:

```
[
  0.13,
 -2.4,
  4.2,
  ...
]
```

These are **logits**.

Then:

$$
P_i=
\frac{e^{z_i}}
{\sum_j e^{z_j}}
$$

using softmax.

---

20. Sampling algorithms

The model doesn't necessarily choose:

```
argmax(probability)
```

Instead, generation can use:

Greedy decoding

```
choose highest probability token
```

Temperature

Modify logits:

$$
P_i=
softmax(z_i/T)
$$

Lower T:

```
more deterministic
```

Higher T:

```
more diverse
```

Top-K

Only consider the K highest-probability tokens.

```
100,000 tokens
      ↓
```

```
top 50
      ↓
sample
```

Top-P / nucleus sampling

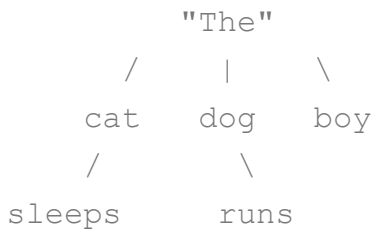
Select the smallest set of tokens whose cumulative probability exceeds P.

---

21. Beam search

Beam search keeps multiple candidate sequences.

Example:



Keep the best B candidates.

This is a classic search/priority-queue problem.

However, beam search is much more common in some sequence-generation applications than in typical open-ended chat LLM decoding.

---

22. KV cache

This is one of the most important LLM-serving concepts.

Suppose the model generates:

```
A B C D E
```

When generating `E`, naively recomputing attention for:

```
A B C D
```

would waste computation.

Instead, cache previous:

```
K
V
```

values.

```
Token A ⟶ K₁,V₁
Token B ⟶ K₂,V₂
Token C ⟶ K₃,V₃
Token D ⟶ K₄,V₄
```

Then when generating E:

```
Q₅
 │
 ├── K₁,V₁
 ├── K₂,V₂
 ├── K₃,V₃
 └── K₄,V₄
```

This dramatically improves autoregressive inference.

But it introduces a major memory problem:

> **KV cache can consume enormous GPU memory.**

This leads to serving techniques such as:

- paged attention
- prefix caching
- KV-cache quantization
- continuous batching
- cache eviction

---

23. Paged attention

Paged attention treats KV cache somewhat like virtual memory.

Instead of requiring one large contiguous memory region:

```
Request A
████████████████████
```

we divide cache into blocks:

```
Block 1
Block 2
Block 3
Block 4
```

and maintain mappings:

```
Logical sequence
      ↓
Block table
      ↓
Physical GPU memory
```

This reduces memory fragmentation and improves GPU utilization.

This is a beautiful example of how **operating-system concepts appear in LLM serving**.

---

24. RAG from a DSA perspective

Retrieval-Augmented Generation introduces another major algorithmic area.

Pipeline:

```
Documents
   ↓
Chunking
   ↓
Embedding
   ↓
Vector database
   ↓
```

```
ANN search
   ↓
Relevant chunks
   ↓
Prompt
   ↓
LLM
```

The central DSA problem is:

> Find the nearest vectors efficiently.

Brute force:

$$
O(Nd)
$$

for N vectors of dimension d.

With millions/billions of vectors, that's expensive.

---

25. HNSW

You specifically want to understand HNSW, and it is highly relevant.

HNSW:

Hierarchical Navigable Small World graph

Conceptually:

```
Level 3:

A -------- D
 \        /
  \      /
    G

Level 2:

A --- C --- D
```

```
  \     \   /
   \     G
```

Higher levels contain fewer nodes.

Search:

```
start at top
   ↓
greedily move toward target
   ↓
drop down one level
   ↓
continue
   ↓
nearest neighbors
```

This is essentially a graph-search problem.

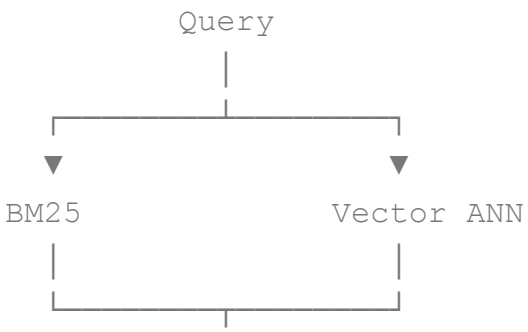
---

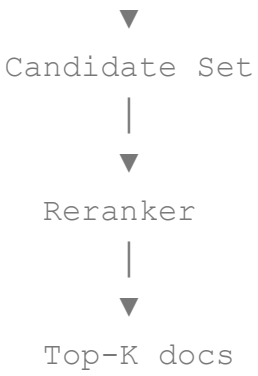
26. Hybrid search

Production RAG frequently combines:

```
BM25
+
Vector similarity
+
Metadata filtering
+
Reranking
```

For example:





This is where classical information retrieval meets LLM systems.

---

27. LLD perspective

Now move one abstraction level higher.

The LLD question is:

> **How would I structure the code that implements an LLM inference system?**

A simplified design:

```
LLMEngine
 ├── Tokenizer
 ├── Model
 │    ├── Embedding
 │    ├── TransformerStack
 │    │    ├── TransformerBlock
 │    │    │    ├── Attention
 │    │    │    ├── FeedForward
 │    │    │    └── Normalization
 │    │    └── ...
 │    └── LMHead
 │
 ├── KVCache
 ├── Scheduler
 ├── Sampler
 └── DeviceManager
```

---

28. Core interfaces

You could model:

```
interface Tokenizer {
    List<Integer> encode(String text);
    String decode(List<Integer> tokens);
}
```

Model:

```
interface LanguageModel {
    Tensor forward(
        Tensor inputIds,
        KVCache cache
    );
}
```

Sampler:

```
interface Sampler {
    int sample(Tensor logits, SamplingConfig config);
}
```

Cache:

```
interface KVCache {
    void put(RequestId id, LayerId layer, KV value);
    KV get(RequestId id, LayerId layer);
    void evict(RequestId id);
}
```

Scheduler:

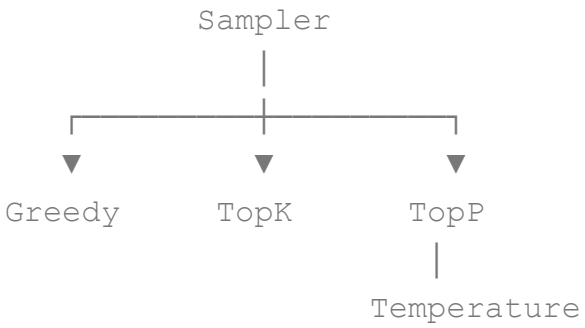
```
interface Scheduler {
    List<Request> schedule(List<Request> requests);
}
```

This naturally leads to SOLID principles.

---

29. Strategy pattern for decoding

Different decoding algorithms can implement the same interface.



For example:

```
interface Decoder {
    Token decode(Logits logits);
}
```

Implementations:

```
GreedyDecoder
TopKDecoder
TopPDecoder
BeamSearchDecoder
```

This is a classic **Strategy Pattern**.

---

30. Model architecture as Composite

A Transformer is naturally hierarchical.

```
Transformer
   ├── Block
   │     ├── Attention
   │     ├── FFN
   │     └── Norm
   ├── Block
```

```
    │   ├── Attention
    │   ├── FFN
    │   └── Norm
    └── ...
```

This resembles the **Composite Pattern**.

---

31. Pipeline pattern

Inference itself forms a pipeline:

```
Request
  ↓
Validation
  ↓
Tokenization
  ↓
Scheduling
  ↓
Prefill
  ↓
Decode
  ↓
Sampling
  ↓
Detokenization
  ↓
Streaming Response
```

Each stage can have clear responsibilities.

---

32. Prefill vs decode

This distinction is crucial.

Prefill

Process the entire input prompt.

```
"Explain distributed systems"
              ↓
       process all tokens
```

This is highly parallelizable.

Decode

Generate one token at a time.

```
token 1
  ↓
token 2
  ↓
token 3
  ↓
token 4
```

Decode is much more latency-sensitive.

Therefore:

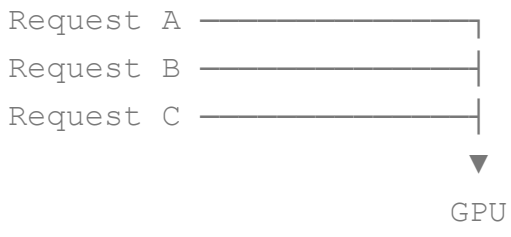
```
Inference
 ├── Prefill
 └── Decode
```

should be considered separately in performance engineering.

---

33. Continuous batching

Traditional batching:



Problem:

```
A finishes
B finishes later
C finishes much later
```

GPU resources can be poorly utilized.

Continuous batching dynamically inserts and removes requests:

```
Time →

A A A A
B B B B B
C C C
    D D D D
       E E E
```

The scheduler continuously modifies the active batch.

This is a queue/scheduling problem.

---

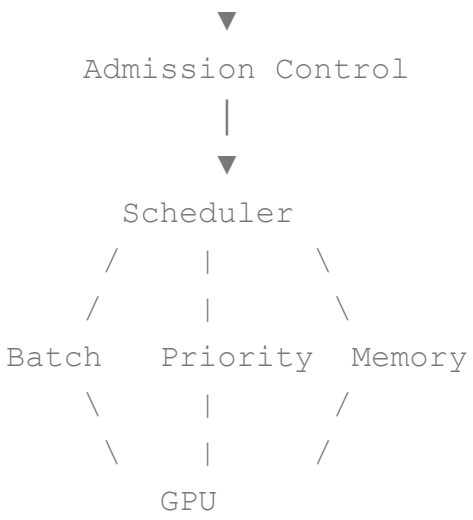
34. LLM scheduler

A production inference scheduler must consider:

```
request arrival
prompt length
max output tokens
KV-cache availability
GPU memory
priority
latency SLO
batch size
fairness
```

You can think of it as:

```
                Request Queue
                     |
```



---

35. Streaming

Users don't want to wait 20 seconds for the complete answer.

Instead:

```
HTTP request
      ↓
Generate token
      ↓
send token
      ↓
Generate token
      ↓
send token
      ↓
...
```

Typical protocols:

- HTTP streaming
- Server-Sent Events
- WebSockets
- HTTP/2
- HTTP/3/QUIC in some architectures

For an interview, understand the trade-offs.

---

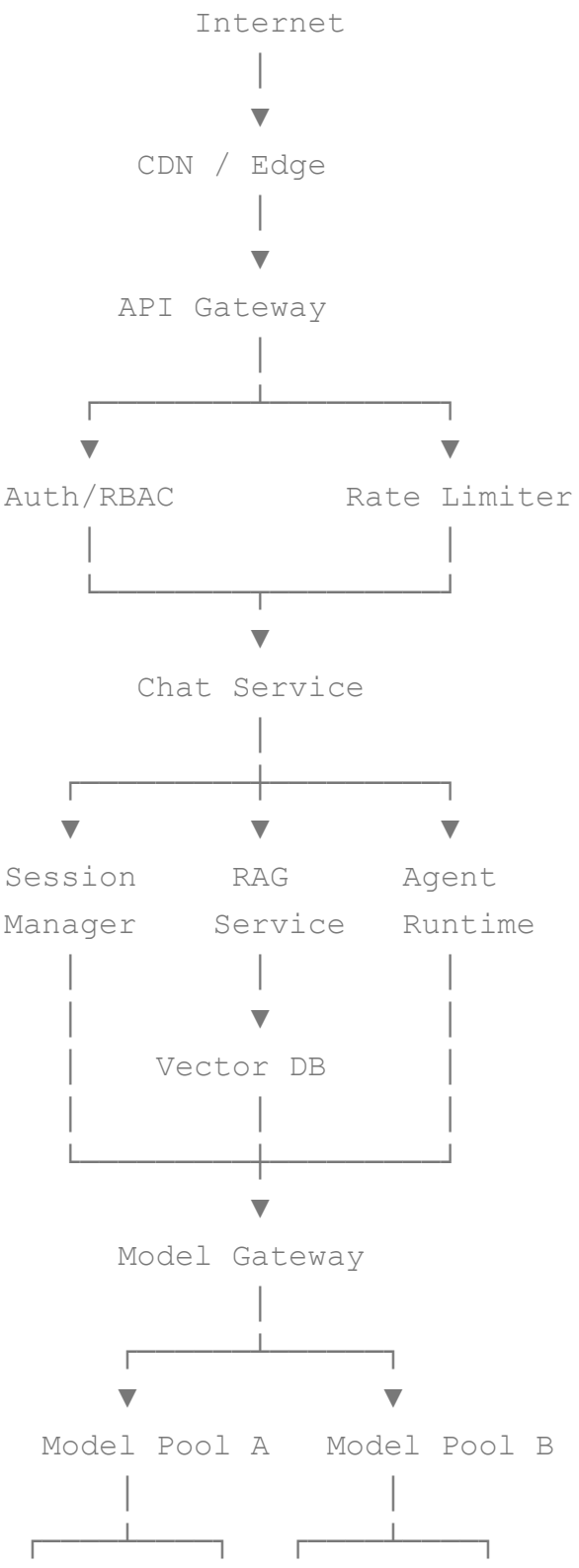
36. HLD perspective

Now we reach the most important level for system-design interviews.

Imagine:

> Design ChatGPT-like infrastructure serving 100 million users.

The architecture could look like:



```
  ▼        ▼  ▼        ▼
GPU      GPU GPU      GPU
```

---

37. Model gateway

A model gateway abstracts the underlying models.

For example:

```
Client
  ↓
Model Gateway
  ├── Small model
  ├── Large model
  ├── Reasoning model
  ├── Embedding model
  └── Vision model
```

It can implement:

- routing
- authentication
- quotas
- retries
- fallback
- load balancing
- model selection
- observability
- cost tracking

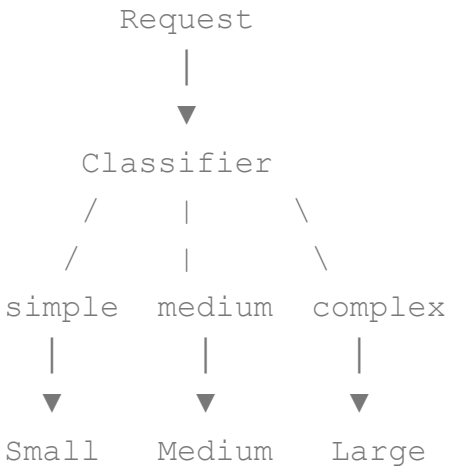
This becomes particularly important when serving multiple models.

---

38. Model routing

You don't necessarily want every request to hit your largest model.

Conceptually:



Routing can consider:

```
task type
latency requirement
context size
complexity
cost
availability
```

---

39. GPU cluster

Large models may not fit on a single GPU.

Suppose:

```
Model = 400 GB
GPU = 80 GB
```

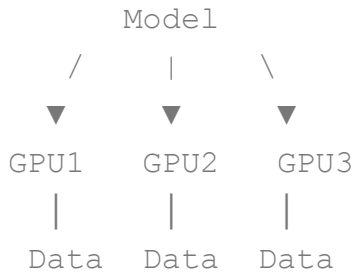
You need multiple GPUs.

Several parallelism strategies become relevant.

---

40. Data parallelism

Each GPU holds a copy of the model.

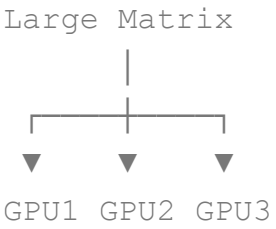


Useful primarily for training.

---

41. Tensor parallelism

Split individual tensor operations across GPUs.



The GPUs cooperate to compute a layer.

This is particularly important for large-model inference/training.

---

42. Pipeline parallelism

Split layers across GPUs:

```
GPU1
Layers 1-20
   ↓
GPU2
Layers 21-40
   ↓
GPU3
Layers 41-60
   ↓
```

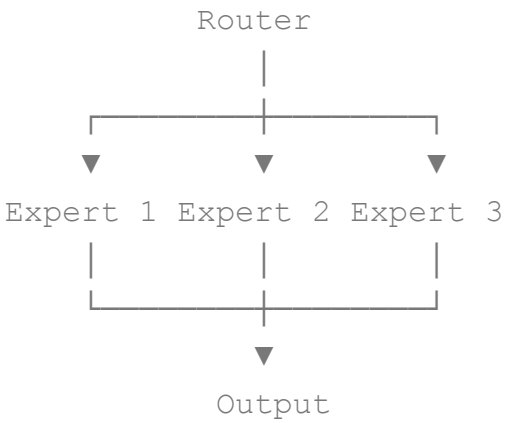
```
GPU4
Layers 61-80
```

The challenge is pipeline bubbles and communication.

---

43. Expert parallelism

For Mixture-of-Experts models:



Only a subset of experts is activated for each token.

This allows:

```
large total parameter count
```

while keeping:

```
active computation
```

lower than a dense model of equivalent total size.

---

44. MoE routing

For each token:

```
token
  ↓
router
  ↓
expert scores
  ↓
top-k experts
  ↓
expert computation
  ↓
combine outputs
```

This introduces a distributed routing problem.

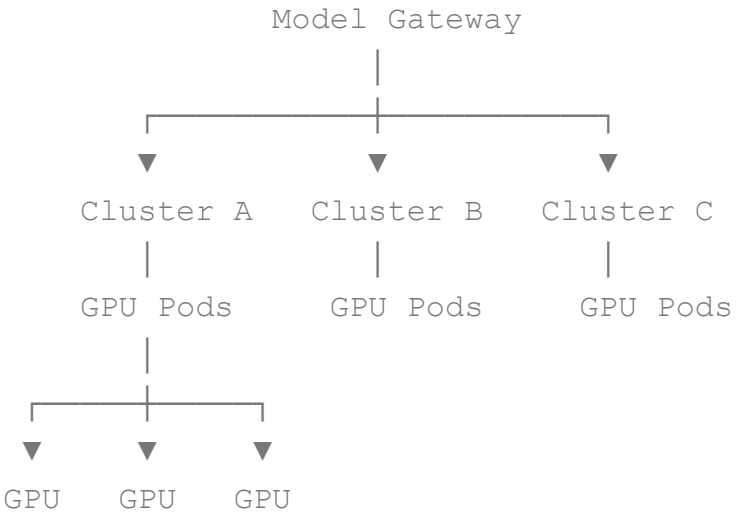
Important concerns include:

- load balancing
- expert capacity
- token dropping
- all-to-all communication
- network bandwidth

---

45. Distributed inference

A production model server might look like:



Load balancing needs to consider more than CPU utilization.

For LLMs:

```
GPU memory
KV-cache usage
active sequences
tokens/sec
queue depth
prompt length
model residency
```

are often more meaningful.

---

46. Why ordinary load balancing is insufficient

Imagine:

```
GPU 1: 20% CPU
GPU 2: 20% CPU
```

They may appear healthy.

But:

```
GPU 1: 95% KV cache
GPU 2: 30% KV cache
```

GPU 1 may be unable to accept another long-context request.

Therefore LLM-aware routing is required.

---

47. Model serving metrics

Important metrics include:

TTFT

Time To First Token

```
request
   ↓
     800 ms
   ↓
first token
```

TPOT

Time Per Output Token

Example:

```
50 ms/token
```

Tokens/sec

```
20 tokens/sec
```

End-to-end latency

```
network
+
queue
+
prefill
+
decode
+
serialization
```

---

48. LLM capacity planning

Traditional services might use:

```
requests/sec
```

LLM systems need:

```
tokens/sec
```

because:

```
request A = 100 tokens
request B = 10,000 tokens
```

These requests are radically different workloads.

Therefore:

$$
TPS = Tokens\ Per\ Second
$$

is a critical capacity metric.

---

49. Token budget

Suppose:

```
1 million requests/day
```

Average:

```
input = 1,000 tokens
output = 500 tokens
```

Total:

$$
1,000,000\times1,500

1.5B
$$

tokens/day.

Average throughput:

$$
\frac{1.5B}{86400}
\approx17,361
$$

tokens/sec.

Peak traffic might be 5× or 10× higher.

This gives you a first approximation of infrastructure requirements.

---

50. Quantization

Large models require substantial memory.

Suppose:

```
70B parameters
```

FP16:

$$
70B\times2 bytes
\approx140GB
$$

INT8:

$$
70B\times1 byte
\approx70GB
$$

INT4:

$$
70B\times0.5 byte
\approx35GB
$$

There are additional metadata and runtime-memory costs, but this demonstrates why quantization is important.

Common concepts:

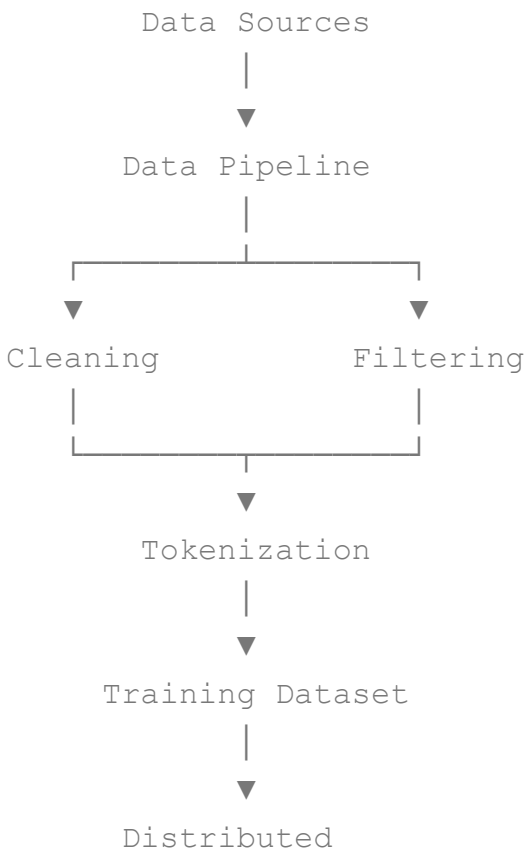
- FP16
- BF16
- INT8
- INT4
- GPTQ
- AWQ
- SmoothQuant

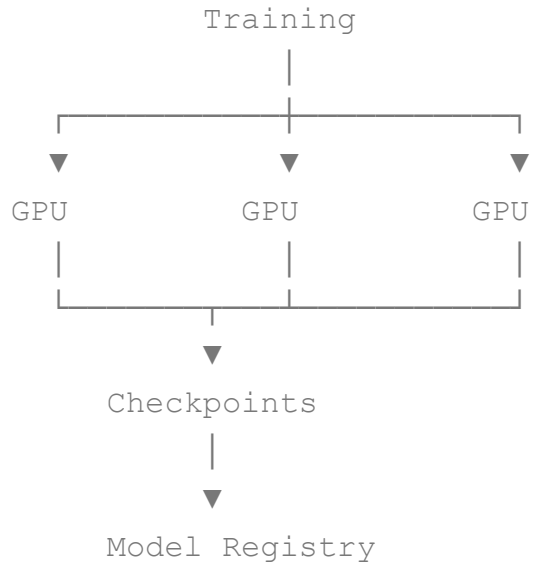
---

51. Training architecture

Inference is only half the story.

A large-scale training system:





---

52. Pretraining

The basic objective is next-token prediction.

Training example:

```
Input:
The capital of France is

Target:
Paris
```

More generally:

$$
L=-\sum_t \log P(x_t|x_{<t})
$$

This is cross-entropy loss.

Training minimizes this loss using gradient-based optimization.

---

53. Backpropagation

The training loop:

```
Forward pass
     ↓
Prediction
     ↓
Loss
     ↓
Backpropagation
     ↓
Gradients
     ↓
Optimizer
     ↓
Updated parameters
```

Optimizers include:

```
SGD
Adam
AdamW
```

Adam/AdamW are particularly important for Transformer training.

---

54. Distributed training

A large model may require thousands of GPUs.

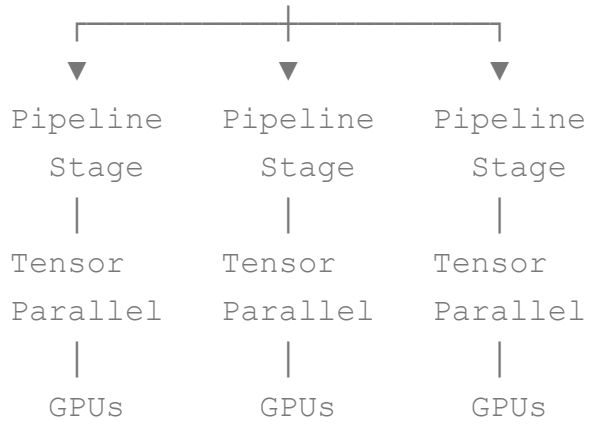
You need:

```
data parallelism
tensor parallelism
pipeline parallelism
expert parallelism
```

Often these are combined.

For example:

```
                Cluster
                   |
```



---

55. Checkpointing

Training can run for days or weeks.

Hardware failure is inevitable.

Therefore checkpoints are periodically written:

```
Training
   │
   ├── checkpoint-1000
   ├── checkpoint-2000
   ├── checkpoint-3000
   └── checkpoint-4000
```

If a worker fails:

```
restore latest checkpoint
        ↓
resume training
```

This is a classic distributed-systems durability problem.

---

56. Fault tolerance

At large scale:

```
GPU failure
node failure
rack failure
network failure
storage failure
```

must be expected.

Strategies:

- checkpointing
- replication
- elastic training
- worker restart
- failure detection
- topology-aware scheduling
- redundant metadata
- durable object storage

---

57. Data pipeline architecture

Training data can come from:

```
web
books
code
papers
licensed datasets
synthetic data
human-generated data
```

Pipeline:

```
Raw Data
   ↓
Deduplication
   ↓
Quality Filtering
   ↓
Safety Filtering
```

```
    ↓
PII Filtering
    ↓
Language Detection
    ↓
Normalization
    ↓
Tokenization
    ↓
Sharding
    ↓
Training Dataset
```

This is essentially a massive distributed data-processing system.

---

58. Fine-tuning

After pretraining:

```
Base Model
     ↓
Supervised Fine-Tuning
     ↓
Instruction-Tuned Model
```

For example:

```
Question
    ↓
Desired answer
```

Training teaches the model how to follow instructions.

---

59. Preference optimization

Another stage can optimize responses according to preferences.

Historically:

```
SFT
 ↓
Reward Model
 ↓
RLHF
```

Other approaches include:

```
DPO
GRPO
RLAIF
```

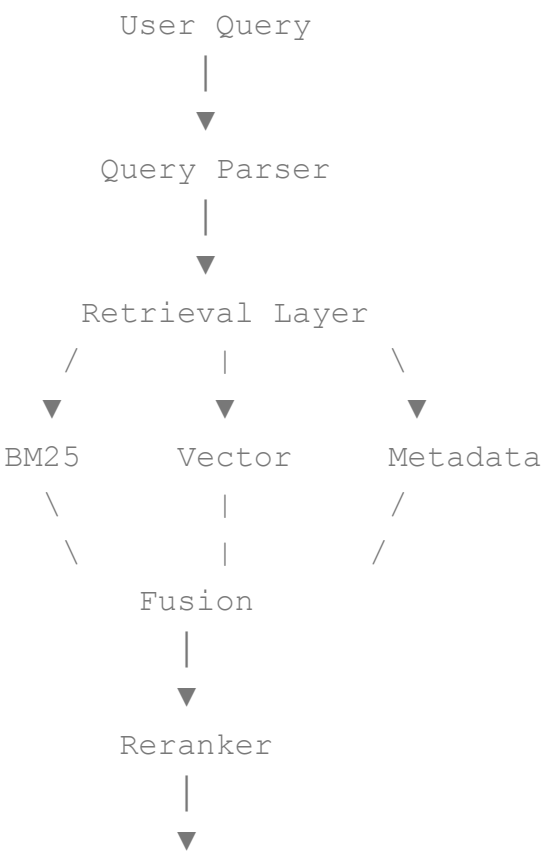
The exact algorithm varies by model and training pipeline.

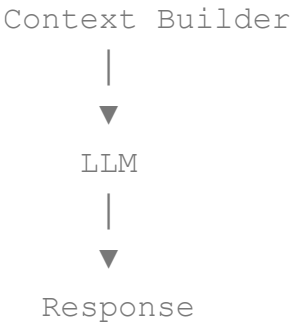
---

60. RAG architecture

A production AI application often shouldn't put all knowledge into model weights.

Instead:





This separates:

```
reasoning/generation
```

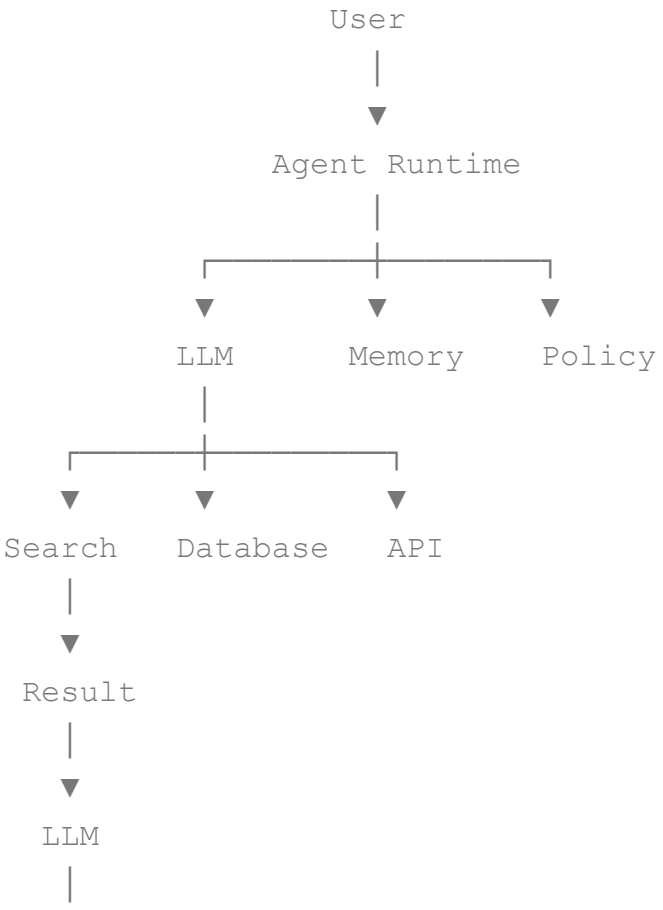
from:

```
external knowledge
```

---

61. Agent architecture

Now add tools.





This becomes an **agentic system**, rather than merely an LLM.

---

62. MCP and tool protocols

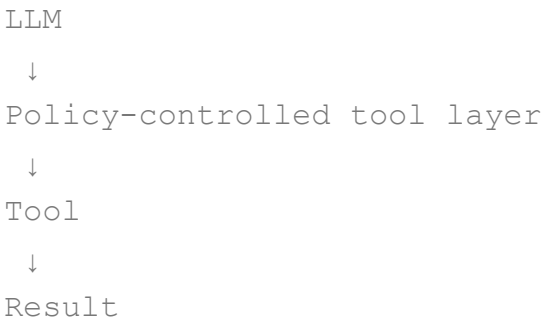
For an AI platform, you can introduce standardized tool interfaces:



The key architectural principle is:

> The LLM should not directly own infrastructure credentials or arbitrary execution privileges.

Instead:



This provides isolation and governance.

---

63. Security architecture

An enterprise LLM platform should include:

```
Identity
Authorization
Secrets
Network isolation
Prompt injection defense
Data loss prevention
PII detection
Tool authorization
Audit logging
Rate limiting
Tenant isolation
```

A useful architecture:

```
Client
  ↓
API Gateway
  ↓
AuthN/AuthZ
  ↓
Policy Engine
  ↓
Agent Runtime
  ↓
Tool Gateway
  ↓
External System
```

Don't let:

```
LLM-generated text
```

be treated as trusted executable instructions.

---

64. Multi-tenancy

Suppose:

```
Customer A
Customer B
Customer C
```

share the same infrastructure.

You need isolation for:

```
conversation
documents
embeddings
vector indexes
KV cache
logs
billing
models
tools
secrets
```

Tenant ID should propagate throughout the request:

```
Request
  ↓
tenant_id
  ↓
Gateway
  ↓
RAG
  ↓
Vector DB
  ↓
LLM
  ↓
Audit
```

---

65. Caching

LLM systems can have several cache layers.

Prompt cache

```
same prefix
    ↓
reuse computation
```

KV cache

```
reuse attention K/V
```

Semantic cache

```
similar question
       ↓
cached answer
```

Retrieval cache

```
query
 ↓
cached retrieval result
```

Embedding cache

```
document
 ↓
cached embedding
```

Caching can dramatically reduce cost and latency.

---

66. Observability

Traditional metrics aren't sufficient.

You need:

Infrastructure

```
CPU
GPU utilization
GPU memory
network
disk
```

Serving

```
TTFT
TPOT
tokens/sec
queue latency
batch size
KV-cache utilization
```

Quality

```
groundedness
retrieval recall
answer relevance
hallucination rate
tool success rate
```

Cost

```
input tokens
output tokens
GPU seconds
```

```
model cost
tenant cost
```

---

67. Distributed tracing

A single request might traverse:

```
API Gateway
   ↓
Agent
   ↓
Retriever
   ↓
Vector DB
   ↓
Reranker
   ↓
LLM Gateway
   ↓
GPU cluster
   ↓
Tool
   ↓
LLM
```

Distributed tracing should preserve:

```
trace_id
span_id
tenant_id
request_id
model_id
```

This is especially important for debugging latency.

---

68. Reliability

An LLM system has many failure points:

```
Client
 ↓
Gateway
 ↓
Auth
 ↓
RAG
 ↓
Vector DB
 ↓
Model Gateway
 ↓
Scheduler
 ↓
GPU
 ↓
Tool
```

You need:

```
timeouts
retries
circuit breakers
bulkheads
fallback models
load shedding
backpressure
rate limits
health checks
```

But retries must be carefully designed.

For example, retrying an expensive generation request can make overload worse.

---

69. Backpressure

Suppose:

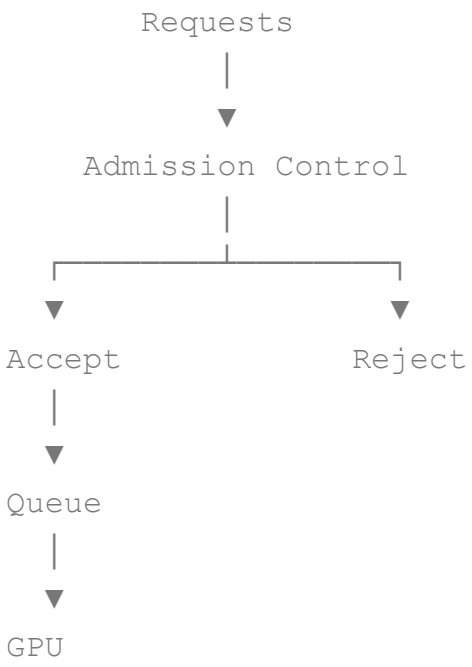
```
Incoming:
1000 requests/sec

GPU capacity:
700 requests/sec
```

If you simply accept everything:

```
queue
queue
queue
queue
OOM
latency explosion
```

Instead:



Possible mechanisms:

- bounded queues
- token budgets
- priority queues
- admission control
- load shedding

---

70. Context window

An LLM has a maximum context length.

For example:

```
Input:
100,000 tokens
```

The model may support:

```
128K context
```

But longer context isn't free.

Attention and KV cache can become expensive.

Therefore production systems use:

```
chunking
summarization
retrieval
compression
sliding windows
prefix caching
```

---

71. Long-context architecture

Instead of sending:

```
10 million tokens
```

to the LLM:

```
10M documents
      ↓
retrieval
```

```
      ↓
100 relevant chunks
      ↓
reranking
      ↓
10 relevant chunks
      ↓
LLM
```

This is a fundamental systems-design principle:

> **Retrieve relevant information rather than blindly increasing context.**

---

72. LLM system bottlenecks

You should be able to identify bottlenecks quickly.

Layer	Typical bottleneck
Tokenizer	CPU
Prefill	GPU compute
Decode	GPU memory bandwidth
KV cache	GPU memory
RAG	Vector DB / network
Reranking	CPU/GPU
Agent	External APIs
Training	GPU compute / network
MoE	Interconnect / all-to-all
Storage	Checkpoint I/O

This distinction is very important.

---

73. Latency decomposition

For an LLM request:

$$
T_{total}

T_{network}
+
T_{queue}
+
T_{prefill}
+
T_{decode}
+
T_{postprocess}
$$

Where:

```
T_queue
```

can dominate during overload.

This is why capacity planning and scheduling are as important as model optimization.

---

74. Cost decomposition

Similarly:

$$
Cost =
GPU\ Cost
+
Storage
+

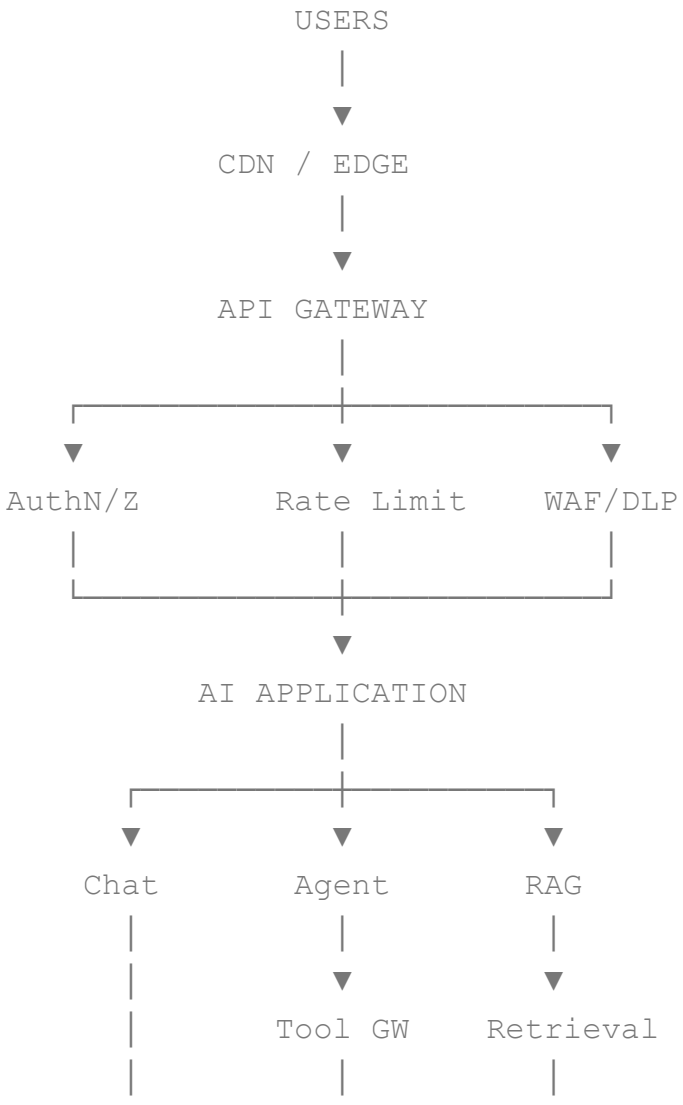
Network
+
VectorDB
+
Reranker
+
Observability
+
Tool\ Calls
$$

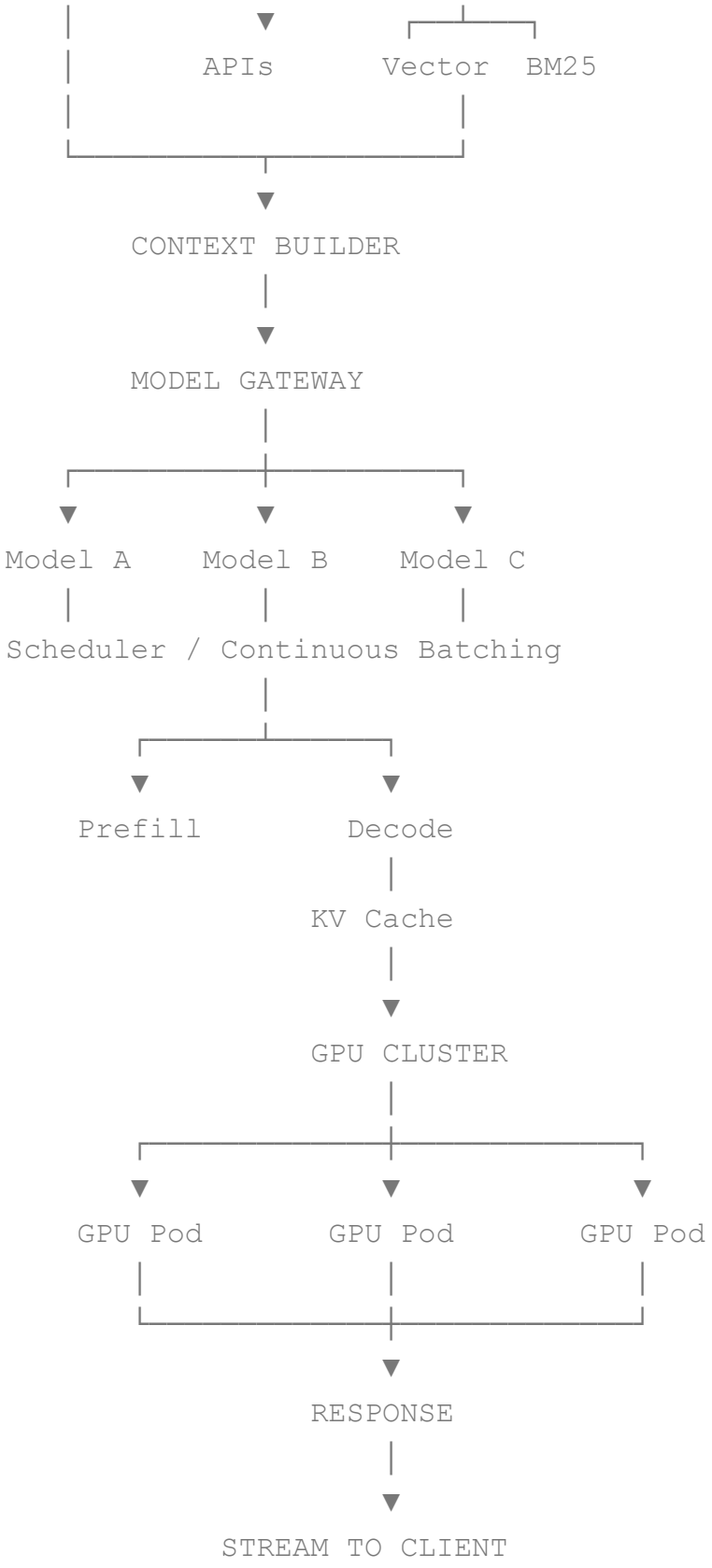
At scale, model inference usually dominates, but the architecture determines how much unnecessary inference occurs.

---

75. A complete production architecture

Putting everything together:





Supporting Systems:

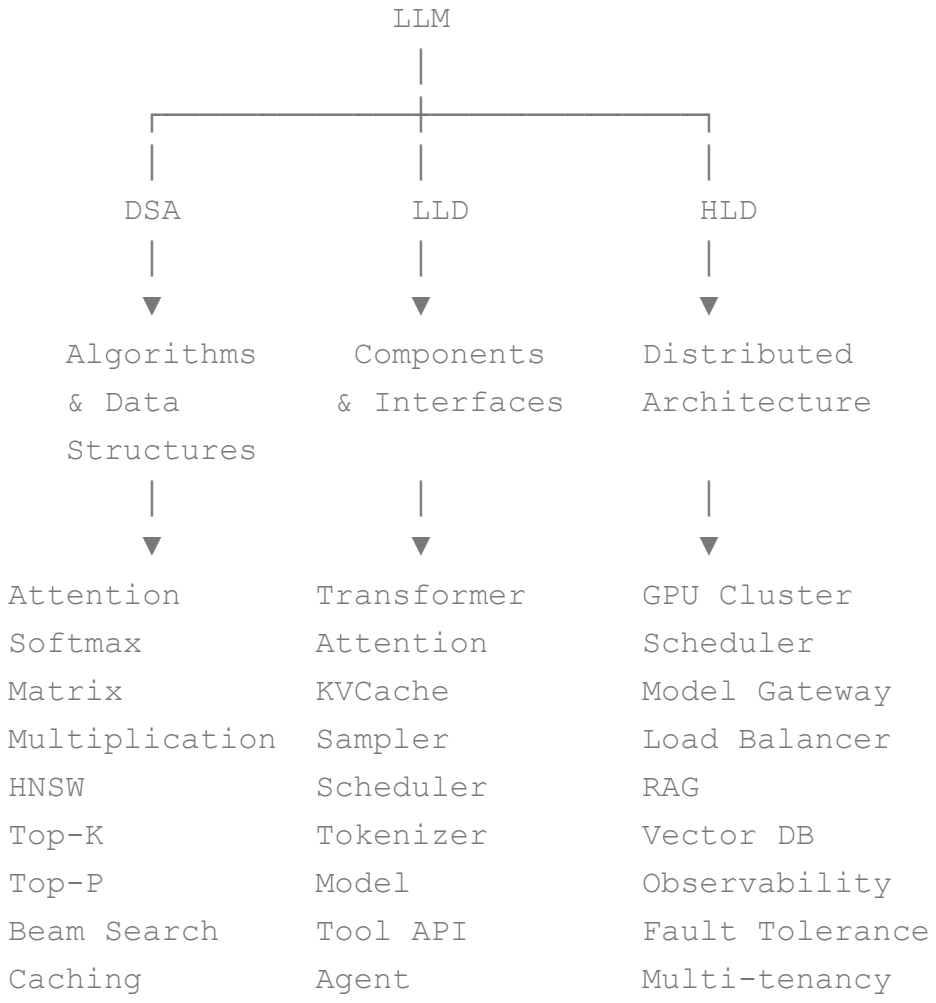
Model Registry
Prompt Registry
Feature/Config Store
Evaluation Platform

```
Dataset Pipeline
Checkpoint Storage
Observability
Billing
Audit
Security
```

---

76. How DSA, LLD and HLD connect

This is the key mental model I recommend.



The three levels aren't independent.

For example:

DSA

```
KV cache
```

LLD

```
KVCacheManager
```

HLD

```
distributed GPU inference platform
```

Likewise:

DSA

```
HNSW
```

LLD

```
VectorIndex
Retriever
Reranker
```

HLD

```
distributed RAG platform
```

This connection is exactly what you should practice for senior/staff-level interviews.

---

77. What you should know for MAANG+ interviews

I would organize your LLM preparation into **10 layers**.

Layer 1 — Mathematics

Know:

```
vectors
matrices
dot products
matrix multiplication
probability
softmax
cross entropy
gradients
backpropagation
optimization
```

Layer 2 — DSA

Know:

```
hash tables
tries
graphs
priority queues
search
ANN
HNSW
top-K
sampling
caching
scheduling
```

Layer 3 — Transformer

Know extremely well:

```
tokenization
embeddings
Q/K/V
self-attention
```

```
causal masking
multi-head attention
RoPE
RMSNorm
FFN
SwiGLU
residual connections
logits
sampling
```

Layer 4 — Model architectures

Understand:

```
decoder-only
encoder-only
encoder-decoder
dense models
MoE
multimodal models
reasoning models
```

Layer 5 — Inference

Know:

```
prefill
decode
KV cache
paged attention
continuous batching
speculative decoding
quantization
prefix caching
tensor parallelism
pipeline parallelism
```

Layer 6 — Training

Know:

```
pretraining
SFT
RLHF
DPO
distributed training
checkpointing
data parallelism
tensor parallelism
pipeline parallelism
expert parallelism
```

Layer 7 — RAG

Know:

```
chunking
embedding
vector DB
HNSW
BM25
hybrid search
reranking
metadata filtering
context construction
evaluation
```

Layer 8 — Agents

Know:

```
planning
tool calling
memory
MCP
A2A
workflow orchestration
state machines
human-in-the-loop
sandboxing
```

Layer 9 — Production architecture

Know:

```
API Gateway
Model Gateway
GPU Scheduler
Load Balancer
Rate Limiting
Caching
Observability
Multi-tenancy
Security
Fault tolerance
Cost optimization
```

Layer 10 — AI system design

Be able to design:

```
ChatGPT
Copilot
RAG platform
AI search engine
AI coding assistant
agent platform
LLM gateway
model-serving platform
vector database
LLM evaluation platform
multimodal AI platform
```

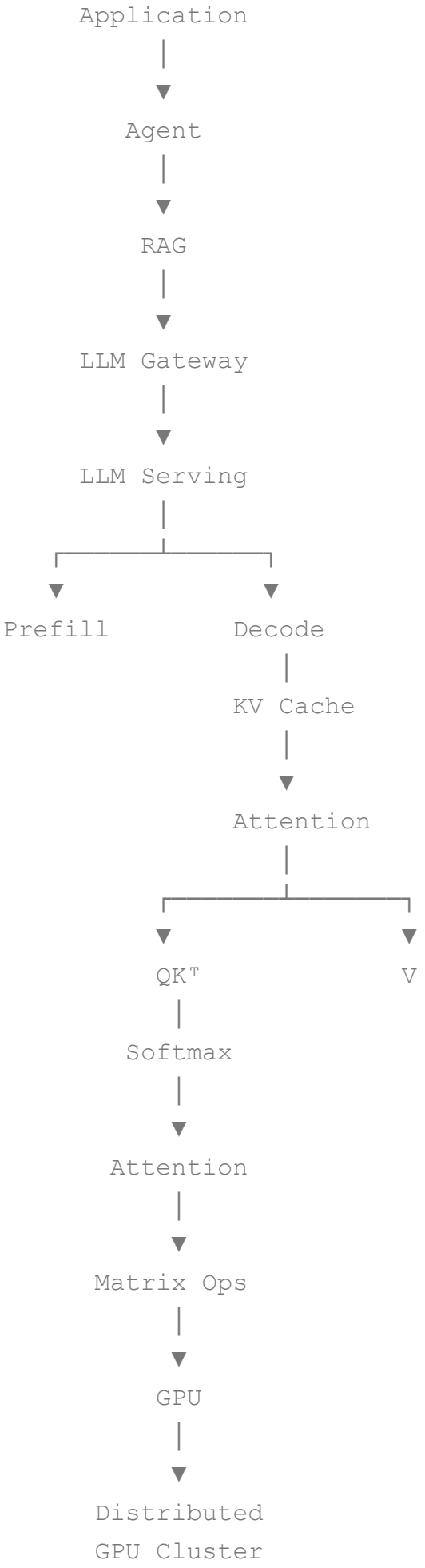
---

78. The most important interview insight

For a senior/staff engineer, don't stop at:

> "How does attention work?"

You should be able to progressively zoom out:



That is the **DSA → LLD → HLD mental model**.

79. A practical learning sequence for you

Given your existing backend, distributed-systems, PostgreSQL, Kafka, Redis, Kubernetes, RAG, vector DB, HNSW, LangChain/LangGraph, MCP and AI-agent work, I would not approach LLMs as a beginner.

I would study them in this order:

```
Week 1
├── Tokenization
├── Embeddings
├── Tensor mathematics
└── Probability / softmax

Week 2
├── Self-attention
├── Multi-head attention
├── Causal masking
├── RoPE
└── Transformer blocks

Week 3
├── Training
├── Backpropagation
├── AdamW
├── SFT
└── Preference optimization

Week 4
├── KV cache
├── Prefill/decode
├── Continuous batching
├── Paged attention
└── Speculative decoding

Week 5
├── Quantization
├── Tensor parallelism
├── Pipeline parallelism
├── Data parallelism
└── MoE
```

Week 6
├── GPU architecture
├── Distributed inference
├── GPU scheduling
├── Model routing
└── Capacity planning

Week 7
├── Embeddings
├── HNSW
├── BM25
├── Hybrid search
└── Reranking

Week 8
├── Production RAG
├── RAG evaluation
├── Context engineering
└── Long-context systems

Week 9
├── Agents
├── Tool calling
├── MCP
├── Memory
└── Agent orchestration

Week 10
├── LLM security
├── Prompt injection
├── Tool isolation
├── Multi-tenancy
└── Governance

Week 11
├── LLM observability
├── Quality evaluation
├── Cost optimization
└── Reliability

Week 12
└── MAANG+ system-design problems

The particularly valuable part for your **AI-enabled Staff Engineer** goal is to repeatedly take every concept through:

```
Algorithm
   ↓
Data structure
   ↓
Class/interface design
   ↓
Concurrency
   ↓
Distributed implementation
   ↓
Failure modes
   ↓
Scalability
   ↓
Observability
   ↓
Security
   ↓
Cost
```

For example, don't merely learn **HNSW**. Learn:

```
HNSW algorithm
      ↓
Graph data structures
      ↓
Index class design
      ↓
Concurrent insertion/search
      ↓
Sharded vector index
      ↓
Replication
      ↓
Rebalancing
      ↓
Failure recovery
      ↓
Recall/latency monitoring
```

```
        ↓
Multi-tenant isolation
```

That is the level of reasoning that turns LLM knowledge into **senior/staff-level system-design capability**.